Project Explanation: Real Estate Platform Backend

Author: Muhammad Teymurlu

Course: Object-Oriented Programming

Assignment: Task 4 (Bina.az Clone)

1. Introduction and Architectural Overview

This project involves designing the backend architecture for a real estate platform capable of managing various property types, calculating dynamic pricing, and determining feature eligibility. To solve this problem, I utilized core Object-Oriented Programming (OOP) principles, including abstraction, inheritance, polymorphism, and interface segregation.

The foundation of the system is the abstract Property class, which serves as a blueprint. Specific property types—ResidentialApartment, CommercialSpace, and LandPlot—inherit from this base class, ensuring shared attributes are managed centrally while allowing each subclass to define its own specific behaviors.

2. Inheritance and Constructor Chaining

To prevent code duplication, common variables such as propertyId, location, area, and price are encapsulated within the Property superclass.

When instantiating a subclass like CommercialSpace, I used constructor chaining via the super() keyword. This passes the base parameters up to the parent Property constructor to be initialized, allowing the subclass constructor to focus strictly on its unique attributes (like businessType and hasParking). This approach ensures a clean, hierarchical initialization process.

3. Abstraction and Method Overriding

Because every property calculates commissions and final prices differently, it didn't make sense to write a generic calculation method in the parent class. Instead, I declared calculateCommission() and calculateFinalPrice() as abstract methods within the Property class.

This enforced a strict contract: every subclass *must* provide its own implementation. Using the @Override annotation, I defined the specific business logic for each class. For example, CommercialSpace applies a 3.5% commission and a flat 500 AZN fee, while LandPlot applies a straightforward 1.5% commission. This guarantees that calling

calculateFinalPrice() on any property will yield the mathematically correct result for that specific type.

4. Bypassing Inheritance Limits with Interfaces

Java restricts classes to single inheritance, meaning a class can only extend one parent. However, certain properties have overlapping but distinct capabilities. Not all properties are eligible for bank mortgages, and not all properties are VIP listings.

To solve this, I decoupled these behaviors into interfaces: MortgageEligible and VipListing.

- I updated ResidentialApartment to implement MortgageEligible, allowing it to provide concrete logic for the standard mathematical amortization formula used to calculate monthly payments.
- I created a VipProperty class that extends Property while implementing VipListing, demonstrating multiple inheritance of type.

Interfaces allowed me to assign specific capabilities only to the classes that needed them without breaking the primary Property family tree.

5. Polymorphism and Dynamic Method Dispatch

In Main.java, I leveraged upcasting to store all instantiated objects (ResidentialApartment, CommercialSpace, LandPlot, VipProperty) inside a single ArrayList<Property>.

By looping through this list, I demonstrated polymorphism. When the program calls p.displayBasicInfo() or p.calculateCommission(), the Java Virtual Machine uses dynamic method dispatch to figure out exactly which subclass version of the method to run at runtime, despite the variable type being a generic Property reference.

Furthermore, to safely access the interface methods without causing runtime crash errors, I utilized the instanceof operator. By checking if (p instanceof MortgageEligible), the program dynamically identifies whether the current property signed the mortgage contract. If true, the object is safely downcasted, and the specific calculateMonthlyPayment() method is triggered.

6. Utilization of the final Keyword

To enforce immutability and protect critical system constants, I utilized the final keyword in two distinct ways:

1. **public static final double VAT_RATE = 0.18;** - Declared in Main.java, this acts as a global, unchangeable constant. Because it is static, it belongs to the class itself

rather than any instance, saving memory. Because it is final, the 18% tax rate cannot be accidentally overwritten by other parts of the program.

2. **public final String PLATFORM_NAME = "Bina.az Clone";** - This ensures that once the platform's name is initialized, it is permanently locked and cannot be modified via setters or subclass manipulation.
3. If it applies to a method, it prevents it from getting overridden by any subclass. Kind of like the final value, but this time for the methods.
4. Finally, final non-access modifier gets used in the classes for preventing inheritance by other classes. It finalizes the class, and completely restricts the extension of it by any other subclass